



PASSO A PASSO

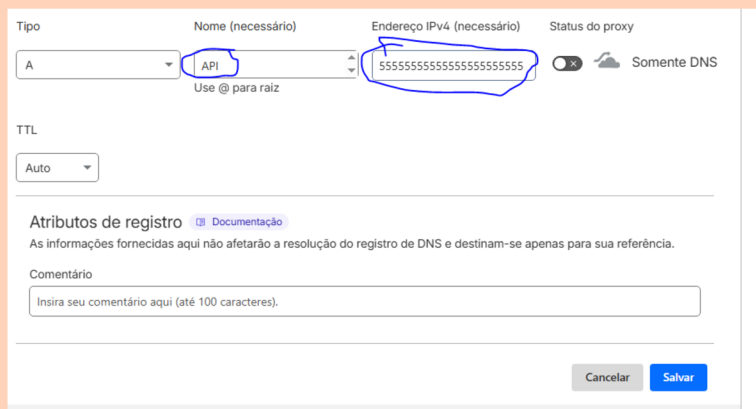
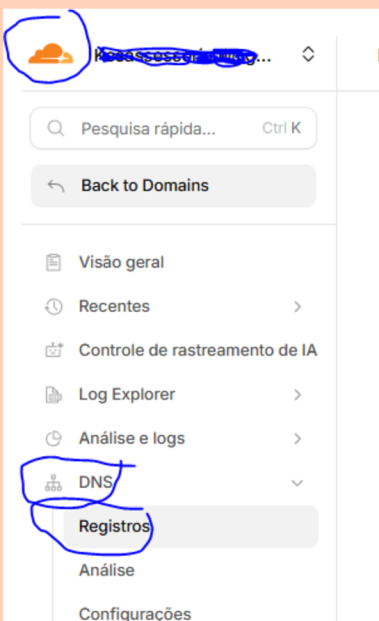
INSTALAÇÃO DO SISTEMA

MULTIATENDIMENTO

ATENDZAPPY

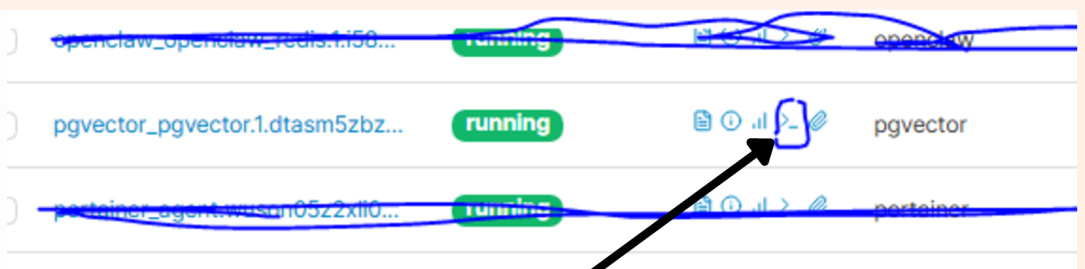
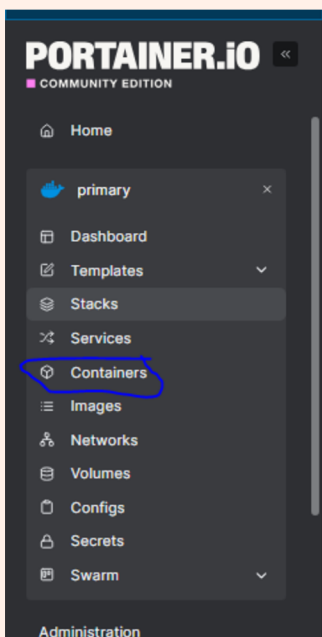
NO PORTAINER

PRIMEIRO PASSO: ENTRE NA CLOUDFLARE E CRIE DOIS SUBDOMÍNIOS – UM API PARA O BACK-END E OUTRO CHAT PARA O FRONT-END.



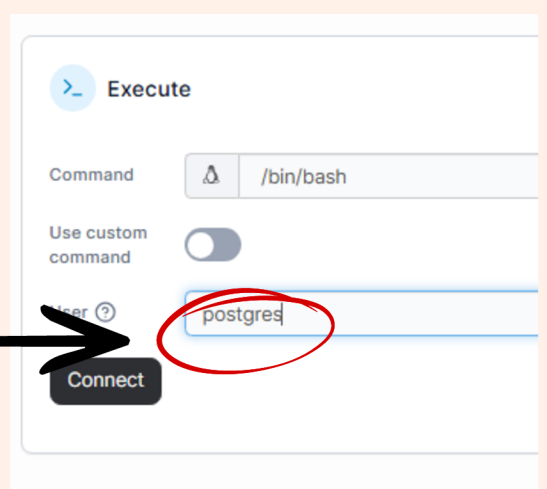
EXEMPLO: API.SEUDOMINIO.COM.BR
EXEMPLO: CHAT.SEUDOMINIO.COM.BR

DEPOIS, ACESSE O SEU PORTAINER E, EM CONTAINERS, PROCURE PELA OPÇÃO PG VECTOR.

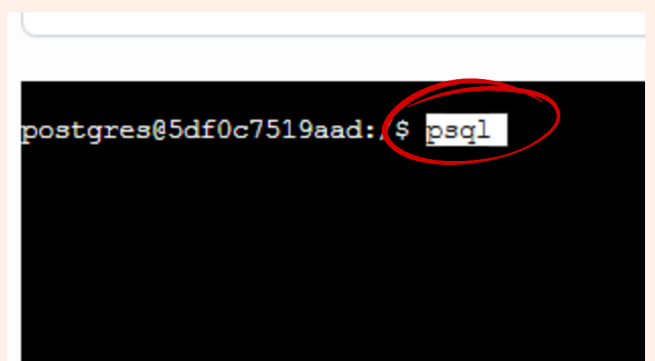


CLIQUE NO BOTÃO CONSOLE.

entra no console do postgres com usuario: postgres



executa: psql



DEPOIS: create database atendzappy;

Para criar o banco de dados dentro do PGVector

```
postgres@5df0c7519aad:/$ psql
psql (16.9 (Debian 16.9-1.pgdg120+1))
Type "help" for help.

postgres=# create database atendzappy;
```

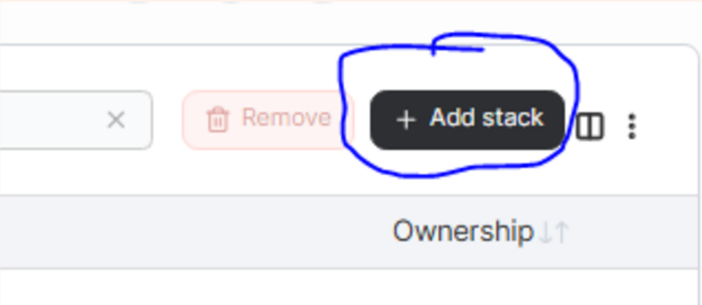
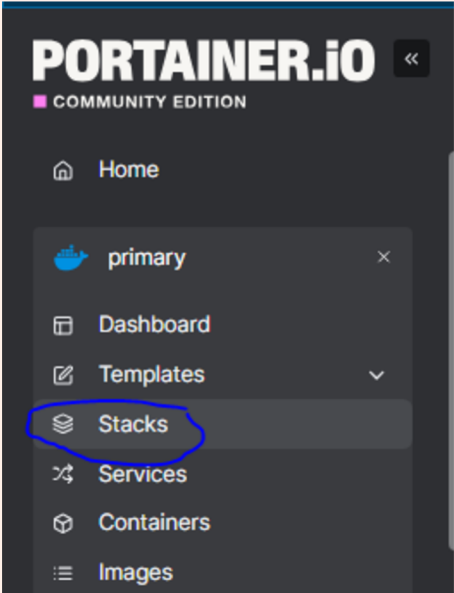
RODE O COMANDO :
ls

PARA listar os bancos E
VERIFICAR SE REALMENTE
O BANCO FOI CRIADO

```
postgres=# \l
```

Name	Owner	Encoding	Locale Pr
atendzappy	postgres	UTF8	libc
chatwoot	postgres	UTF8	libc
postgres	postgres	UTF8	libc
template0	postgres	UTF8	libc
template1	postgres	UTF8	libc

DEPOIS, NO PORTAINER, CLIQUE NA OPÇÃO
STACKS E, EM SEGUIDA, EM ADD STACK.



ADICIONE O NOME ATENDZAPPY, ESCOLHA A OPÇÃO WEB
EDITOR, CONFIGURE A STACK E, EM SEGUIDA, CLIQUE EM
DEPLOY PARA REALIZAR A IMPLANTAÇÃO.

Create stack

Name:

Build method: ☒ Web editor ☐ Upload

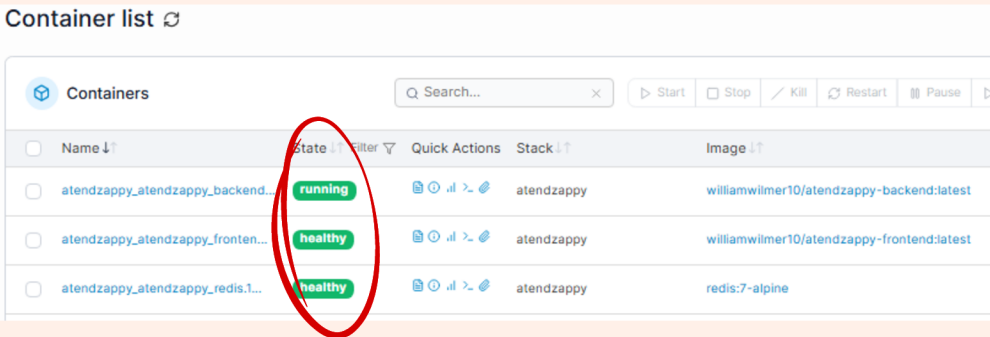
Web editor

Define or paste the content of your docker compose file here

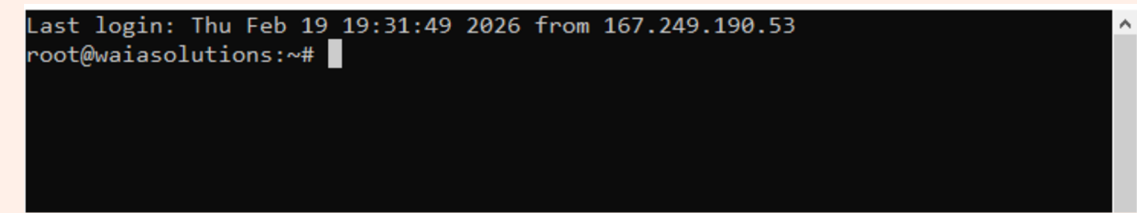
```
1 version: "3.7"
2 services:
3     ## ----- ATENDZAPPY BACKEND -----
4     atendzappy_backend:
```

```
VERSION: "3.7"
SERVICES:
## ----- ATENDZAPPY BACKEND ----- ##
ATENDZAPPY_BACKEND:
  IMAGE: WILLIAMWILMERO/ATENDZAPPY-BACKEND:LATEST
  VOLUMES:
    - ATENDZAPPY_PUBLIC:/APP/PUBLIC
    - ATENDZAPPY_LOGS:/APP/LOGS
  NETWORKS:
    - SUA_REDE
  ENVIRONMENT:
    - NODE_ENV=PRODUCTION
    - PORT=6080
    - TZ=America/Sao_Paulo
    - APP_URL=HTTPS://APICHAAT.SEU_DOMINIO_SITE.COM.BR
    - BACKEND_URL=HTTPS://APICHAAT.SEU_DOMINIO_SITE.COM.BR
    - FRONTEND_URL=HTTPS://CHAT.SEU_DOMINIO_SITE.COM.BR
    - DB_DIALECT=POSTGRES
    - DB_HOST=PGVECTOR
    - DB_PORT=5432
    - DB_USER=POSTGRES
    - DB_PASS=SENHA_DO_SEU_POSTGRE
    - DB_NAME=ATENDZAPPY
    - JWT_SECRET=A1B2C3D4E5F6G7H8J9K1L2PEÇA_O_GFT_PARA_GERAR_O_RESTANTE_DA_CHAVE
    - JWT_REFRESH_SECRET=Z9Y8X7W6V5U4T3PEÇA_O_GFT_PARA_GERAR_O_RESTANTE_DA_CHAVE
    - REDIS_URI=REDIS://ATENDZAPPY123@ATENDZAPPY_REDIS:6379
    - REDIS_OPT_LIMITER_MAX_A
    - CORRIGIDO_REDIS ~ REDIS
    - REDIS_OPT_LIMITER_DURATION=3000
    - USER_LIMIT=999
    - CONNECTIONS_LIMIT=999
    - CLOSED_SIGN_BY_ME=TRUE
    - VERIFY_TOKEN=WHATICKET
    - FACEBOOK_APP_ID=7768
    - FACEBOOK_APP_SECRET=PS45923
    - GOOGLE_CLIENT_ID=604305494657-VUC32CHUL3R0S0SQ
    - GOOGLE_CLIENT_SECRET=GC98PX-GPJNUS
    - GOOGLE_REDIRECT_URI=HTTPS://APIATENDZAPPY
  # - UAZAPI_ADMIN_TOKEN=ZAWIQWTEKUQ7UBICBUYM
  # - UAZAPI_BASE_URL=HTTPS://FREE.UAZAPI.COM
  # - UAZAPI_WEBHOOK_URL=HTTPS://CHAT
  DEPLOY:
    MODE: REPLICATED
    REPLICAS: 1
    PLACEMENT:
    CONSTRAINTS:
      - NODE_ROLE == MANAGER
    RESOURCES:
    LIMITS:
      CPU: "1"
      MEMORY: 1024M
    LABELS:
      - TRAEFIK_ENABLE=TRUE
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_BACKEND.RULE=HOST(APICHAAT.SEU_DOMINIO_SITE.COM.BR)
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_BACKEND.ENTRYPOINTS=WEBSecure
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_BACKEND.TLS.CERTRESOLVER=LETSENCRYPTRESOLVER
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_BACKEND.PRIORITY=1
      - TRAEFIK_HTTP_SERVICES.ATENDZAPPY_BACKEND.LOADBALANCER.SERVER.PORT=6080
      - TRAEFIK_HTTP_SERVICES.ATENDZAPPY_BACKEND.LOADBALANCER.PASSTHROTHHEADER=TRUE
      - TRAEFIK_HTTP_MIDDLEWARES.ATENDZAPPY_SSLHEADER.HEADERS.CUSTOMREQUESTHEADERS.X-FORWARDED-PROTO=HTTPS
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_BACKEND.MIDDLEWARES=ATENDZAPPY_SSLHEADER
  ## ----- ATENDZAPPY FRONTEND ----- ##
ATENDZAPPY_FRONTEND:
  IMAGE: WILLIAMWILMERO/ATENDZAPPY-FRONTEND:LATEST
  NETWORKS:
    - SUA_REDE_NET
  # ALTERAÇÃO ADICIONADA! CONFIGURAÇÃO DE RUNTIME
  # ESTA SEÇÃO DEFINE A URL DO BACKEND VIA VARIÁVEL DE AMBIENTE, PERMITINDO QUE
  # O DOMÍNIO SEJA ALTERADO DIRETAMENTE AQUI NA STACK SEM PRECISAR RECONSTRUIR A IMAGEM.
  ENVIRONMENT:
    - REACT_APP_BACKEND_URL=HTTPS://APICHAAT.SEU_DOMINIO_SITE.COM.BR
    - NODE_ENV=PRODUCTION
  DEPLOY:
    MODE: REPLICATED
    REPLICAS: 1
    PLACEMENT:
    CONSTRAINTS:
      - NODE_ROLE == MANAGER
    RESOURCES:
    LIMITS:
      CPU: "0.5"
      MEMORY: 1024M
    LABELS:
      - TRAEFIK_ENABLE=TRUE
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_FRONTEND.RULE=HOST(CHAT.SEU_DOMINIO_SITE.COM.BR)
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_FRONTEND.ENTRYPOINTS=WEBSecure
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_FRONTEND.TLS.CERTRESOLVER=LETSENCRYPTRESOLVER
      - TRAEFIK_HTTP_ROUTERS.ATENDZAPPY_FRONTEND.PRIORITY=1
      - TRAEFIK_HTTP_SERVICES.ATENDZAPPY_FRONTEND.SERVICE=ATENDZAPPY_FRONTEND
      - TRAEFIK_HTTP_SERVICES.ATENDZAPPY_FRONTEND.LOADBALANCER.SERVER.PORT=60
      - TRAEFIK_HTTP_SERVICES.ATENDZAPPY_FRONTEND.LOADBALANCER.PASSTHROTHHEADER=TRUE
  ## ----- ATENDZAPPY REDIS ----- ##
ATENDZAPPY_REDIS:
  IMAGE: REDIS:7-ALPINE
  COMMAND:
    - REDIS-Server
    - --appendonly
    - --yes
    - --port
    - "6379"
    - --maxmemory
    - "512M"
    - --maxmemory-policy
    - "allkeys-lru"
    - --requirepass
    - "ATENDZAPPY123"
  HEALTHCHECK:
    TEST: ["CMD", "redis-cli", "-a", "ATENDZAPPY123", "ping"]
    INTERVAL: 10s
    TIMEOUT: 5s
    RETRIES: 3
  VOLUMES:
    - ATENDZAPPY_REDIS/DATA
  NETWORKS:
    - SUA_REDE_NET
  DEPLOY:
    PLACEMENT:
    CONSTRAINTS:
      - NODE_ROLE == MANAGER
    RESOURCES:
    LIMITS:
      CPU: "0.5"
      MEMORY: 512M
  ## ----- VOLUMES ----- ##
VOLUMES:
ATENDZAPPY_PUBLIC:
  EXTERNAL: TRUE
NAME: ATENDZAPPY_PUBLIC
ATENDZAPPY_LOGS:
  EXTERNAL: TRUE
NAME: ATENDZAPPY_LOGS
ATENDZAPPY_REDIS:
  EXTERNAL: TRUE
NAME: ATENDZAPPY_REDIS
## ----- NETWORK ----- ##
NETWORKS:
SUA_REDE_NET:
  EXTERNAL: TRUE
NAME: SUA_REDE_NET
```

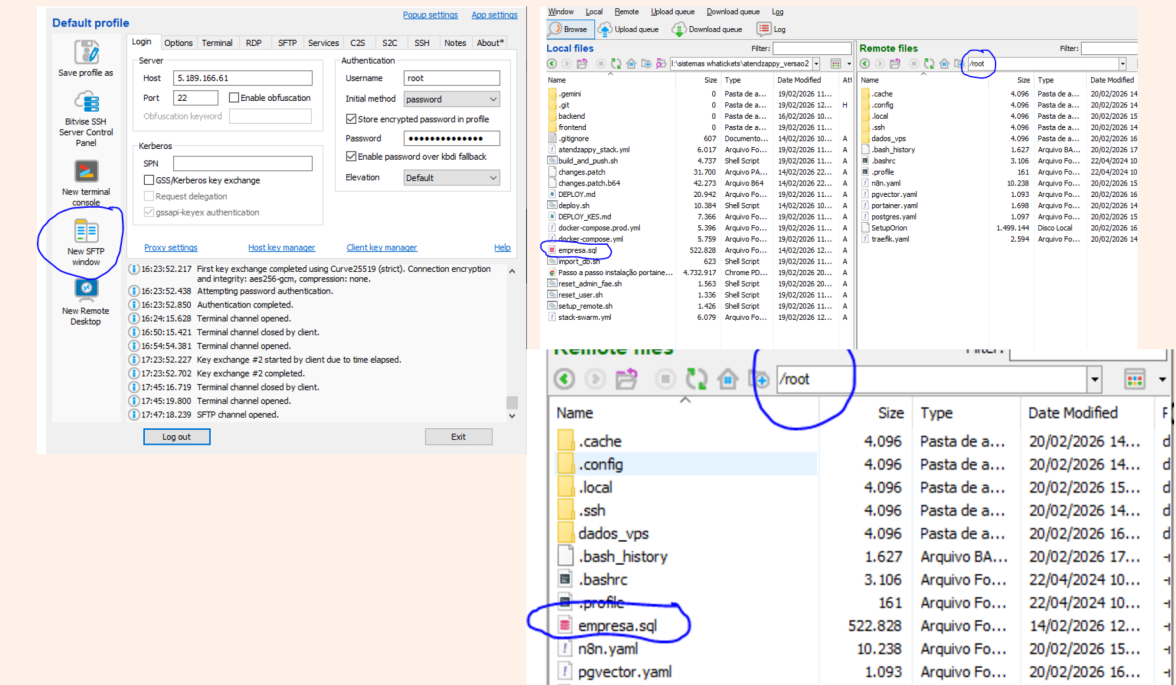
DEPOIS DE REALIZAR O DEPLOY DA STACK, AGUARDE OS CONTÊINERES INICIAREM.



ASSIM QUE ESTIVEREM EM EXECUÇÃO, ACESSE O TERMINAL DO CONTÊINER E EXECUTE OS SEGUINTE COMANDOS:

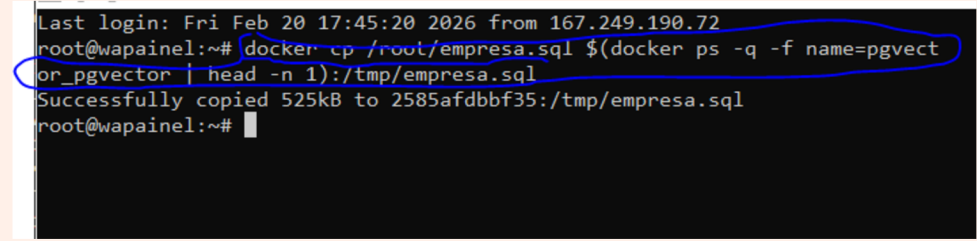


O PASSO 1 É ENVIAR O ARQUIVO EMPRESA.SQL PARA DENTRO DO SERVIDOR. SE VOCÊ ESTIVER UTILIZANDO O BITWISE, CLIQUE NESSA OPÇÃO PARA ACESSAR O FTP.



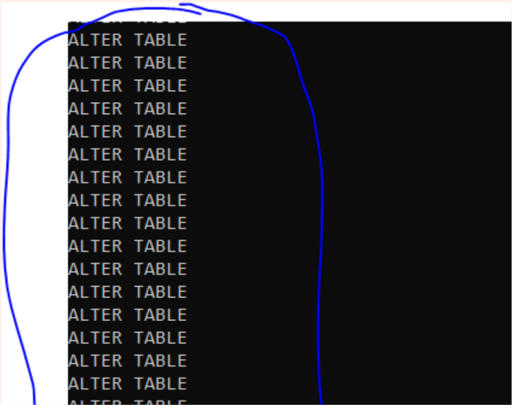
PASSO 2: COLOCAR O SQL DENTRO DO CONTAINER DO BANCO DE DADOS O ARQUIVO JÁ ESTÁ NA SUA VPS (NA PASTA /ROOT). AGORA VAMOS MOVÊ-LO PARA DENTRO DO CONTAINER DO POSTGRES PARA QUE ELE POSSA SER "LIDO". NO TERMINAL DA SUA VPS, RODE:

docker cp /root/empresa.sql \$(docker ps -q -f name=pgvector_pgvector | head -n 1):/tmp/empresa.sql



DEPOIS EXECUTE A IMPORTAÇÃO COM O COMANDO ABAIXO:

docker exec \$(docker ps -q -f name=pgvector_pgvector | head -n 1) psql -U postgres -d atenzappy -f /tmp/empresa.sql

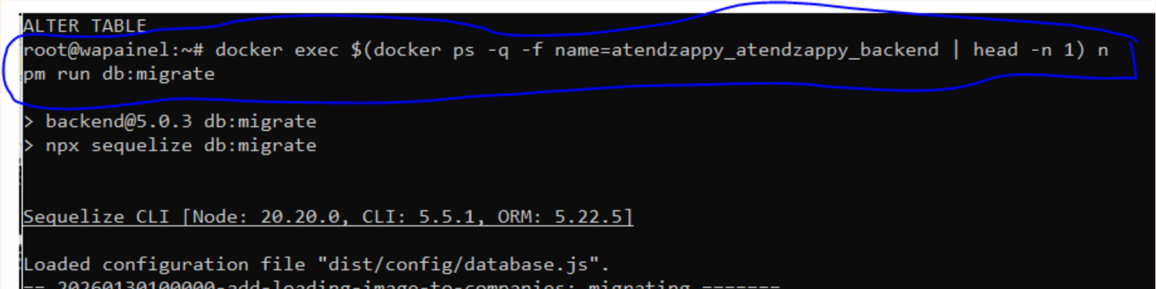


APÓS EXECUTAR TODOS OS COMANDOS E IMPORTAR O ARQUIVO SQL PARA O BANCO DE DADOS, É NECESSÁRIO RODAR O MIGRATE PARA QUE AS ALTERAÇÕES IMPORTADAS SEJAM APLICADAS CORRETAMENTE.

COMO RODAR O MIGRATE NA SUA VPS:
VOCÊ DEVE RODAR O COMANDO DENTRO DO CONTAINER DO BACKEND. COMO VOCÊ ESTÁ USANDO DOCKER SWARM, PRIMEIRO IDENTIFICAMOS O CONTAINER DO BACKEND E DEPOIS EXECUTAMOS O COMANDO.
COMANDO PARA RODAR AGORA (VIA SSH):

O COMANDO FINAL PARA O TUTORIAL:

`docker exec $(docker ps -q -f name=atendzappy_atendzappy_backend | head -n 1) npm run db:migrate`



💎 Tutorial: Recuperação de Acesso Super Admin
Este procedimento ensina como preparar o ambiente e resetar o acesso do administrador principal diretamente pelo terminal da VPS.
Passo 1: Preparação do Diretório
Se você acabou de subir sua VPS e ainda não organizou suas pastas, ou se quer garantir que o local do script esteja correto, siga estes comandos:

- 1. Crie a pasta do projeto (caso não exista):
`mkdir -p /opt/atendzappy`
- 1. O parâmetro `-p` garante que o Linux crie as pastas pai se necessário.
- 1. Acesse a pasta:

`cd /opt/atendzappy`

Passo 2: Criação do Script de Reset
Agora, vamos criar o arquivo que contém a lógica de recuperação.

1. Abra o editor de texto para criar o arquivo:

```
nano reset_admin.sh
```

1. Cole o código abaixo dentro do editor:

```
#!/bin/bash

# 1. Identifica o container do backend (usando um filtro mais flexível)
BACKEND_CONTAINER=$(docker ps -q -f name=atendzappy_backend | head -n 1)

if [ -z "$BACKEND_CONTAINER" ]; then
    echo "❌ Erro: Container do backend não encontrado."
    exit 1
fi

echo "🔒 Gerando hash seguro para a senha '123456'..."
# Gera o hash usando o node do backend
HASH=$(docker exec $BACKEND_CONTAINER node -e "const bcrypt = require('bcryptjs'); console.log(bcrypt.hashSync('123456', 8));")

if [ -z "$HASH" ]; then
    echo "❌ Erro: Falha ao gerar o hash."
    exit 1
fi

# 2. Identifica o container do banco de dados
DB_CONTAINER=$(docker ps -q -f name=pgvector | head -n 1)

if [ -z "$DB_CONTAINER" ]; then
    echo "❌ Erro: Container do banco de dados não encontrado."
    exit 1
fi

# 3. Atualiza o Usuário ID 1 no banco de dados atendzappy
echo "👤 Atualizando super admin para 'user@faedeveloper.com.br'..."

docker exec $DB_CONTAINER psql -U postgres -d atendzappy -c "UPDATE \"Users\" SET email = 'user@faedeveloper.com.br', \"passwordHash\" = '$HASH' WHERE id = 1;"

# 4. Verificação final
echo "🔍 Verificando se o usuário foi atualizado:"
docker exec $DB_CONTAINER psql -U postgres -d atendzappy -c "SELECT id, email FROM \"Users\" WHERE id = 1;"

echo "✅ Acesso resetado com sucesso!"
```

1. **Salve e saia: Pressione**
2. **CTRL+X**
3. **, depois Y**
4. **ENTER**
5. **para salvar, e**
6. **para sair do editor.**

1. **Passo 3: Execução e Permissões**
2. **Para que o script funcione, você precisa dizer ao Linux que ele é um "executável":**
3. **Dê a permissão de execução:**

```
chmod +x reset_admin.sh
```

Rode o script para aplicar as mudanças

```
./reset_admin.sh
```

- **mkdir -p**
- **:** Cria diretórios de forma segura.
- **nano**
- **:** É o editor de texto mais amigável do terminal Linux.
- **chmod +x**
- **:** Transforma um arquivo de texto comum em um programa executável.
- **Hash de Senha:** Aprendemos que sistemas modernos nunca salvam a senha limpa no banco, por isso usamos o script para gerar o "hash" (código embaralhado).